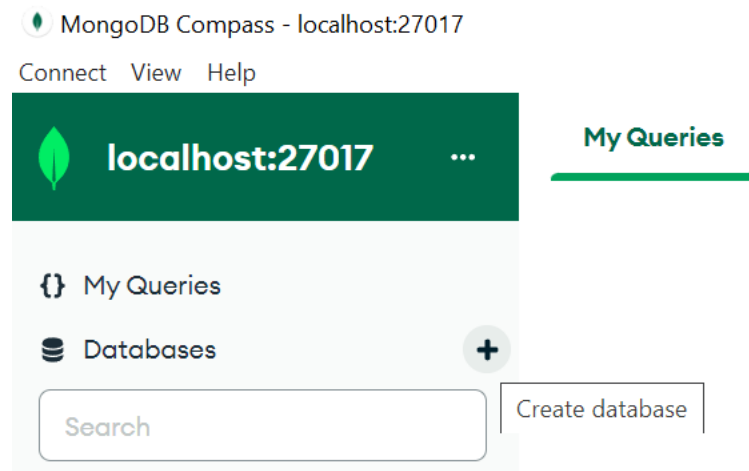


To the cinema with Mongo

A. Database creation

Create a new database called 'Cinema'



Then, in MongoDB compass shell, select the cinema database with '**use cinema;**'

```
>_MONGOSH
> use cinema;
< 'switched to db cinema'
cinema>
```

B. Data insertion

Insert into the Cinema database, the following data representing movies.

```
db.Cinema.insert(
{
  name:"Goodfellas",
  year:1990,
  producer:{surname:"Scorsese", name:"Martin"},
  actors:
  [
```

```
{surname:"De Niro", name:"Robert"},
{surname:"Liotta", name:"Ray"},
{surname:"Pesci", name:"Joe"}
]
})

db.Cinema.insert(
{
name:"The Godfather",
year:1972,
producer:{surname:"Coppola", name:"Francis Ford"},
actors:
[
{surname:"Pacino", name:"Al"},
{surname:"Brando", name:"Marlon"},
{surname:"Duvall", name:"Robert"}
]
})

db.Cinema.insert(
{
name:"Million Dollar Baby",
producer:{surname:"Eastwood", name:"Clint"},
actors:
[
{surname:"Swank", name:"Hilary"},
{surname:"Eastwood", name:"Clint"}
]
})

db.Cinema.insert(
{
name:"Gran Torino",
year:2008,
producer:{surname:"Eastwood", name:"Clint"},
actors:
[
{surname:"Vang", name:"Bee"},
{surname:"Eastwood", name:"Clint"}
]
})
```

```
db.Cinema.insert(
{
  name:"Unforgiven",
  producer:{surname:"Eastwood", name:"Clint"},
  actors:
    [
      {surname:"Hackman", name:"Gene"},
      {surname:"Eastwood", name:"Clint"}
    ]
})

db.Cinema.insert(
{
  name:"Mystic River",
  producer:{surname:"Eastwood", name:"Clint"},
  actors:
    [
      {surname:"Penn", name:"Sean"},
      {surname:"Bacon", name:"Kevin"}
    ]
})

/* show the collection of documents of the database Cinema*/

db.Cinema.find()
```

C. Querying data

1. Which movies were released in 1990?

Reminder

```
db.Cinema.find(JSON document)
db.col.find({"attribute":"value"})
```

Solution

```
db.Cinema.find({"year":1990})
```

2. Which movies were released before 2000?

Reminder

We will use the object `{<lt;:value}` instead of the value of the attribute to be tested (`<lt;` for less than).

Solution

```
db.Cinema.find({"year":{$lt:2000}})
```

3. Which movies did Clint Eastwood direct?

Reminder

We will use an object as a value.

Solution

```
db.Cinema.find({"producer":{"surname":"Eastwood", "name":"Clint"}})
```

4. Which movies did someone named Clint make?

Reminder

Use the object property browser: *point*. *object.attribute*.

Solution

```
db.Cinema.find({"producer.name":"Clint"})
```

5. Which movies did someone named Clint make before 2000?

Reminder

Here, we use a list of *attribute:value* conditions to specify an *AND* (logical AND):

```
db.col.find({"attribute1":"value1", "attribute2":"value2"})
```

Solution

```
db.Cinema.find({"producer.name":"Clint", "year":{$lt:2000}})
```

6. Which movies does Clint Eastwood played in?

Solution

```
db.Cinema.find({"actors":{"surname":"Eastwood", "name":"Clint"}})
```

7. What movies does an Eastwood play in?

Reminder

Use *\$elemMatch*:{"key":"value"} parser instead of value.

Solution

```
db.Cinema.find({"actors":{"$elemMatch":{"surname":"Eastwood"}}})
```

8. What are the names of the movies where an Eastwood played in?

Reminder

To manage the projection, use a second argument of the find () clause:

```
db.Cinema.find({selection JSON document }, {projection JSON document})
```

with projection JSON document of the form: {"attribute1":1, "attribute2":1...}

The identifiers are always displayed by default, if you want to remove them, you can add the `_id:0` clause in the projection document.

Solution

```
/* With identifiers*/
```

```
db.Cinema.find({"actors":{"$elemMatch":{"surname":"Eastwood"}}},{ "name":1})
```

```
/* Without identifiers*/
```

```
db.Cinema.find({"actors":{"$elemMatch":{"name":"Eastwood"}}},{ "name":1, "_id":0}  
)
```

9. What movies does an Eastwood play in?

Reminder

Use `$elemMatch:{"key":"value"}` parser instead of value.

Solution

```
db.Cinema.find({"actors":{"$elemMatch":{"surname":"Eastwood"}}})
```